

UFC Fight Prediction Data Platform

Interview Preparation Guide

A comprehensive explanation of the data pipeline, warehouse, feature engineering, machine learning approach, prediction serving workflow, technical decisions, and likely interview questions.

PROJECT TYPE	End-to-end sports analytics data and ML platform
PRIMARY OUTPUT	Calibrated pre-fight UFC win probabilities
PRODUCTION MODEL	XGBoost with Platt calibration
WAREHOUSE	PostgreSQL loaded from UFCStats-derived CSV snapshots
DASHBOARD CONTRACT	Reporting views for current predictions and post-event performance
GENERATED FROM	README, docs, SQL migrations, modeling code, and project artifacts

Note: Details explicitly marked as assumptions are inferred from repository structure or common deployment practice.

Contents

- 1. Project Overview
 - 2. Data Layer
 - 3. Technology Stack
 - 4. System Architecture
 - 5. Key Technical Decisions
 - 6. Features and Functionality
 - 7. AI, Machine Learning, and Data Processing Logic
 - 8. Challenges and Solutions
 - 9. Interview Q&A Preparation
 - 10. Future Improvements
- Assumptions and interview positioning notes are included where repository details are incomplete.

1. Project Overview

This project is an end-to-end UFC fight prediction data platform. It collects UFCStats data, stores normalized historical records in PostgreSQL, builds leakage-safe pre-fight features, trains and compares machine learning models, produces calibrated win probabilities for upcoming fights, and records post-event performance for honest model review.

Main Goals

- Create a repeatable data pipeline from scraped UFC event, fight, fighter, and statistics pages into a clean analytical warehouse.
- Generate model-ready pre-fight snapshots that only use information available before a bout happens.
- Train probabilistic models that are evaluated with chronological splits, not random splits, to match live prediction conditions.
- Score upcoming fight cards and publish dashboard-friendly reporting views.
- Close the loop by comparing saved pre-event predictions against actual results after an event finishes.

Target Users and Use Case

- Primary users: analysts, portfolio reviewers, and interviewers evaluating a practical data/ML system.
- Dashboard users: people who want a concise view of upcoming UFC picks, confidence tiers, and model accuracy.
- The system is explicitly not designed for live in-fight prediction or automated betting execution.

Key Problems Solved

- Sports data is messy: identifiers, missing stats, future events, and corrected pages require idempotent acquisition and validation.
- Fight prediction is leakage-prone: feature generation must exclude the target fight and any later information.
- Accuracy alone is not enough: predictions are probabilities, so calibration, log loss, Brier score, and uncertainty bands matter.
- Backtests can be misleading: the project preserves saved pre-event predictions so post-event reporting can distinguish real predictions from retroactive scoring.

2. Data Layer

Data Used

- Events: event name, date, location, status, source URL, and card membership.
- Fights: two fighters, winner, weight class, scheduled rounds, title-fight flags, finish method, round/time, referee, and source URL.
- Fighters: identity, nickname, height, weight, reach, stance, date of birth, source URL, and scrape timestamp.
- Fight statistics: aggregate and round-level striking, takedowns, control time, knockdowns, submission attempts, and reversals.
- Derived data: fighter snapshots, bout features, predictions, dashboard views, model metrics, and post-event review reports.

Data Sources

- Primary source: ufcstats.com pages scraped through a Scrapy project under scraper/UFC-Web-Scraping-main.
- Canonical parsed snapshots live under data/*.csv: events.csv, fights.csv, fighters.csv, fight_stats.csv, and fight_stats_by_round.csv.
- Raw HTML captures and fetch manifests are part of the acquisition contract; they allow re-parsing and audit without always re-fetching the site.
- Assumption: the public dashboard consumes PostgreSQL reporting views from this backend or a synchronized database, because the dashboard repository is separate.

Main Entities and Storage

Layer	Entity	Grain	Purpose
Raw	HTML pages	one file per page	Source audit and re-parseability
CSV	events/fights/fighters/stat	source-specific rows	Portable snapshots and loader input
Warehouse	events, fights, fighters	normalized records	Reliable joins and constraints
Warehouse	fight_stats_aggregate	fighter-fight	Fight-level performance totals
Warehouse	fight_stats_by_round	fighter-round	Round-level stat detail
Features	fighter_snapshots	fighter-fight	Pre-fight state at cutoff date
Features	bout_features	fight	Model-ready pairwise features
Serving	predictions	fight-score time	Prediction history and post-event

Collection, Storage, Processing, Retrieval

- Collection: Scrapy spiders crawl event, fight, fighter, aggregate-stat, and round-stat pages. Queue CSVs drive incremental fighter and stats fetches.
- Storage: CSV snapshots are loaded into PostgreSQL using warehouse loader scripts. Loaders use typed transforms and batch upserts.
- Processing: feature modules compute career, rolling, decayed, physical, Elo, opponent-adjusted, and debut-prior features.
- Retrieval: modeling code reads bout_features through psycopg2 into pandas. Dashboard consumers read views such as current_event_predictions and pre_event_prediction_events.
- Validation: warehouse integrity and consistency scripts check joins, completeness, and data sanity. Feature tests assert no leakage.

Data Assumptions and Limitations

- UFCStats does not provide injuries, camp quality, weight-cut problems, private betting market information, or reliable short-notice replacement context.
- Nationality is not collected because it is not available from UFCStats in the scraper overview.
- Early fights may have incomplete or missing stats tables; the pipeline must tolerate missing aggregate or round-level records.
- Both-debuting fights contain very little historical signal and are treated as high-uncertainty cases.
- The fighter ordering comes from source data; models predict P(fighter_1 wins), so positional bias is monitored as a limitation.

3. Technology Stack

Area	Technology	Why It Was Chosen
Scraping	Scrapy	Async crawling, spider structure, middleware, queues
Data work	Python	Single language for ETL, features, modeling, CLI
Frames	pandas, NumPy	Fast tabular transforms and model matrices
Warehouse	PostgreSQL	Relational joins, constraints, views, reporting contract
DB access	psycopg2	Direct SQL control and efficient batch inserts
Modeling	scikit-learn	Baselines, calibration, metrics, train/test utilities
Modeling	XGBoost, LightGBM	Strong tabular nonlinear models
Explainabilit	SHAP	Feature attribution and error/uncertainty analysis
Testing	pytest	Unit and integration tests for loaders, features, models
Orchestration	Makefile	Simple reproducible command interface
Analysis	Jupyter notebooks	Exploration, model comparison, saved-prediction review
Visualization	Power BI + web dashboa	Separate presentation layer for predictions

Frontend Technologies

This repository is primarily the backend/data/ML system. The README states that the public dashboard is maintained separately in wlodzimierrr/ufc-dashboard and should read from PostgreSQL reporting views. This repository also contains Power BI assets, so dashboarding is treated as a downstream presentation layer rather than mixed into the ML pipeline.

Backend and Data Technologies

- Python modules separate warehouse loading, feature computation, modeling, and prediction CLI responsibilities.
- PostgreSQL is used because the data is naturally relational: events have fights, fights have two fighters and stats rows, and reporting views can join predictions back to outcomes.
- CSV remains in the acquisition layer because it is easy to diff, inspect, load with pandas, and preserve as a source snapshot.
- Make targets encode common workflows: migrate, load_all, warehouse_check, build_features, train_all_v2, predict_pipeline, review_event, and pre_event_log.

4. System Architecture

High-Level Architecture

UFCStats pages

- > Scrapy spiders + parsers + RawCaptureMiddleware
- > data/*.csv + data/raw/ + data/manifests/
- > warehouse loaders + transforms + migrations
- > PostgreSQL normalized warehouse
- > feature pipeline: fighter_snapshots + bout_features
- > model training/comparison/calibration artifacts
- > upcoming feature builder + scoring CLI
- > predictions table + dashboard reporting views
- > dashboard / Power BI / post-event review reports

Main Components and Responsibilities

- scraper/: discovers source pages, captures raw HTML, parses records, builds queue and coverage reports.
- warehouse/: owns migrations, connection handling, CSV transforms, upserts, and integrity checks.
- features/: computes historical pre-fight snapshots and bout-level features with explicit leakage prevention.

- modeling/: loads feature data, creates temporal splits, trains candidates, calibrates probabilities, compares metrics, scores upcoming fights, and reviews events.
- predict.py: user-facing CLI that scores all upcoming fights, a named event, or the next event; saves CSV and database prediction history.
- docs/: project contracts, runbooks, feature catalog, data dictionary, model card, and implementation notes.

Request/Response and Batch Flow

- This is mostly a batch-oriented system, not a request-per-user API. A scoring run loads the production model pointer, reads upcoming features, predicts probabilities, applies calibration, assigns confidence tiers, and upserts predictions.
- Dashboard reads are SQL view reads: current_event_predictions exposes latest upcoming predictions; pre_event_prediction_fights/events compare predictions saved before event day with actual outcomes.
- Post-event refresh re-scrapes completed data, reloads the warehouse, rebuilds upcoming features, and re-scores predictions for future events.

Authentication and Authorization

- No application-level authentication flow is implemented in this repository.
- Database access is protected through environment variables loaded from .env: POSTGRES_HOST, POSTGRES_PORT, POSTGRES_DB, POSTGRES_USER, and POSTGRES_PASSWORD.
- Assumption: production access control is handled by database credentials, hosting platform permissions, and dashboard deployment configuration outside this repository.

5. Key Technical Decisions

Leakage-Safe Feature Snapshots

- Decision: build fighter_snapshots using a cutoff equal to each event date, then combine two snapshots into one bout_features row.
- Reasoning: prevents target fight stats, later fights, or future profile changes from leaking into training examples.
- Trade-off: more complex feature pipeline and tests, but much more credible evaluation.

Chronological Splits Instead of Random Splits

- Decision: train/validation/test are split by event_date; rolling temporal CV is used for model selection.
- Reasoning: real predictions always happen forward in time, so validation should mimic deployment.
- Trade-off: less data in each split and potentially noisier metrics, but fewer inflated results.

PostgreSQL Warehouse Plus CSV Snapshots

- Decision: keep source snapshots as CSV and load normalized tables into PostgreSQL.
- Reasoning: CSVs are portable and auditable; PostgreSQL provides constraints, joins, indexes, upserts, and dashboard views.
- Trade-off: two storage representations to maintain, offset by clearer separation between acquisition and analytics.

XGBoost with Calibration as Production Model

- Decision: production_model.json points to XGBoost (calibrated), selected for lowest test log loss: 0.6318.
- Reasoning: fight predictions are probability products, so log loss and calibration matter more than raw accuracy alone.
- Trade-off: ensemble had higher accuracy but worse log loss; the selected model is better for calibrated probabilities.

Prediction History Rather Than Overwriting

- Decision: predictions table primary key is (fight_id, scored_at), preserving multiple scores for the same fight.
- Reasoning: enables honest post-event audits using predictions saved before event_date.
- Trade-off: reporting needs latest/pre-event selection logic, handled with SQL views.

Performance, Scalability, Security, Maintainability

- Performance: batch processing with pandas and SQL is appropriate for UFC-scale data: thousands of fights, tens of thousands of stat rows.
- Scalability: the architecture can scale by partitioning prediction history, adding scheduled jobs, and materializing expensive views if read traffic grows.
- Security: credentials are environment-based; no database secrets are hardcoded. Public outputs should expose predictions, not raw credentials or scraper internals.
- Maintainability: modules are separated by layer, Makefile targets document workflows, and tests cover transforms, features, leakage, and modeling artifacts.

6. Features and Functionality

Main User-Facing Features

- Scrape and refresh UFC historical and upcoming data.
- Load normalized event, fight, fighter, and statistics tables into PostgreSQL.
- Build historical features and upcoming fight features.
- Train and compare Logistic Regression, LightGBM, XGBoost, ensemble, Elo, favorite, and coin-flip baselines.
- Score upcoming cards and output table or JSON predictions through predict.py.
- Save predictions to models/predictions and to the PostgreSQL predictions table.
- Review completed events and build pre-event prediction logs.
- Serve dashboard views for upcoming predictions, fight-level review, event-level accuracy, and fighter summaries.

How Key Features Work Technically

- Incremental scraping: spiders use existing IDs and queue files to skip already captured records and append only new information.
- Warehouse upsert: loader rows are deduplicated by primary key before INSERT ... ON CONFLICT updates existing records.
- Feature pipeline: career, rolling, decay, physical, Elo, opponent, debut-prior, and bout modules create reusable feature families.
- Upcoming scoring: build_upcoming.py creates model columns for future fights; score_upcoming.py loads the production model, fills missing feature columns with NaN, predicts, calibrates, and assigns tiers.
- Post-event review: completed fights are joined to saved predictions and actual outcomes to compute correctness, accuracy, log loss, and Brier score.

Important User Flows

```
Historical rebuild:  
make migrate -> make load_all -> make warehouse_check -> make build_features -> make train_all_v2
```

```
Upcoming predictions:  
make load_upcoming -> make build_upcoming_features -> make predict
```

```
Post-event review:  
make post_event -> make review_event EVENT="UFC 315" -> make pre_event_log
```

Edge Cases Handled

- Draws and no-contests are excluded from labeled modeling rows because the target is binary.
- Debuting fighters receive sparse-history handling and debut-prior features; both-debuting bouts are considered very uncertain.
- Missing physical fields are preserved as nulls with missingness flags where appropriate.
- Upcoming fights may have missing labels; they are stored separately from historical labeled training rows.
- Fights with missing stats pages do not crash the whole acquisition run; coverage reports expose gaps.
- Re-scoring the same fight creates a new prediction row instead of losing the previous score.

7. AI, ML, and Data Processing Logic

Modeling Problem

The ML task is binary probabilistic classification: predict whether fighter_1 defeats fighter_2. The output is not just a pick; it is a calibrated probability used for confidence tiers and post-event scoring.

Feature Families

- Career aggregates: fight count, wins/losses, finish rates, title experience, cage time, striking, grappling, and defense.
- Rolling windows: last 1, 3, and 5 fight summaries for form, pace, accuracy, defense, control, knockdowns, submissions, and streaks.
- Decay features: alpha=0.85 exponentially weighted metrics so recent performance matters more.
- Physical/activity: age, age squared, height, reach, reach-to-height ratio, stance flags, days since last fight, long layoff, short-notice proxy, debut flag.
- Elo/opponent strength: sequential Elo with K=32, opponent average Elo, and opponent-adjusted significant-strike accuracy.
- Bout features: fighter_1 minus fighter_2 differences, bounded ratio features, stance matchup flags, weight-class encoding, and debut priors.

Model Selection and Validation

Metric	Production XGBoost calibrated
-----	-----
Test log loss	0.6318
Test accuracy	62.9%
Test AUC	0.703
Test Brier score	0.2209
Expected calibration error	0.0614
Train rows	6,376
Validation rows	1,007
Test rows	1,017

- Training period: earliest through 2022-03-11.
- Validation period: 2022-03-12 through 2024-03-08.
- Test period: 2024-03-09 onward.
- Calibration: Platt scaling on validation predictions, applied to future scores.
- Selection rationale: lowest test log loss, even though the stacked ensemble had slightly higher accuracy.

Confidence and Limitations

- High confidence: $p \leq 0.30$ or $p \geq 0.70$; 29.0% of test fights; 68.7% test accuracy.
- Medium: 0.30-0.40 or 0.60-0.70; 34.1% of test fights.
- Toss-up: 0.40-0.60; 36.9% of test fights; near coin-flip accuracy around 52.0%.

- Known weak spots include both-debuting fights, title fights, missing injury/camp information, no betting-market signal, and weak discrimination in some divisions.

8. Challenges and Solutions

Challenge: Data Leakage

- Problem: post-fight stats or future fights can silently inflate model performance.
- Solution: use `as_of_date` cutoffs, exclude target fights from histories, order Elo sequentially, and test leakage behavior.
- Interview framing: this is the most important architectural decision because credible sports prediction depends on temporal discipline.

Challenge: Messy Source Data

- Problem: source pages can change, stats may be missing, and identity fields may be incomplete.
- Solution: raw HTML capture, fetch manifests with hashes, deterministic IDs, coverage reports, review CSVs, and tolerant null handling.

Challenge: Sparse Fighter Histories

- Problem: debuting or low-history fighters have few prior stats, making rolling and career features sparse.
- Solution: preserve nulls for models that handle them, add debut flags and priors, and expose both-debuting fights as uncertain.

Challenge: Choosing the Right Metric

- Problem: accuracy can reward overconfident but poorly calibrated models.
- Solution: choose by log loss and calibration quality, report Brier score/ECE, and keep confidence tier analysis.

Challenge: Honest Post-Event Evaluation

- Problem: retroactive backtests can be confused with predictions actually made before events.
- Solution: predictions table preserves `scored_at`; `pre_event_prediction` views only count predictions where `scored_at::date < event_date`.

What You Learned

- Data engineering choices dominate ML credibility in time-dependent prediction systems.
- A simple baseline and a calibrated model can be more valuable than a complex ensemble if the product is probability-focused.
- Versioned artifacts, model cards, and runbooks make the system explainable to both technical and non-technical audiences.

9. Interview Q&A Preparation

Q: What does this project do?

A: It turns UFCStats data into calibrated pre-fight win probabilities. The pipeline scrapes and stores data, builds pre-fight features, trains and calibrates models, scores upcoming fights, and reviews accuracy after events.

Q: Why did you build a warehouse instead of modeling directly from CSV?

A: The normalized PostgreSQL warehouse gives reliable joins, constraints, upserts, indexes, and reporting views. CSVs remain useful as source snapshots, but the warehouse is better for repeatable feature generation and dashboard queries.

Q: How do you prevent data leakage?

A: Every fighter snapshot is computed at a cutoff equal to the fight date and excludes the target fight and later bouts. Temporal splits are chronological, Elo is updated sequentially, and leakage tests validate the behavior.

Q: Why use XGBoost as the production model?

A: It had the best test log loss after calibration, which matters because the product is a probability. The ensemble had higher accuracy but worse log loss, so XGBoost was the better calibrated decision.

Q: What is Platt calibration doing here?

A: The model first outputs raw probabilities. Platt scaling fits a logistic transformation on validation predictions, then applies that transformation to future predictions so probabilities better match observed frequencies.

Q: Why not random train/test splits?

A: Random splits let later-era information bleed into earlier-era evaluation and make the task easier than deployment. Chronological splits better represent predicting future fights from prior history.

Q: How does the system handle debut fighters?

A: Debut fighters have sparse history, so the pipeline uses debut flags and priors, preserves missing values, and confidence logic tends to place both-debuting fights into toss-up territory.

Q: What are the core model features?

A: Elo, career record, striking and grappling efficiency, defense, recent form, decayed performance, age/reach/stance, activity, title/five-round experience, opponent strength, and pairwise differences/ratios.

Q: How do upcoming predictions flow through the system?

A: Load upcoming fights, build upcoming features, load the production model pointer, predict raw probabilities, fit/apply calibration, assign confidence tiers, save CSV outputs, and upsert prediction history into PostgreSQL.

Q: How does the dashboard get data?

A: The dashboard should read PostgreSQL views such as `current_event_predictions` for live cards and `pre_event_prediction_events/fights` for post-event accuracy. The frontend is intentionally separate from this backend repo.

Q: What would you say is the hardest engineering problem?

A: Maintaining temporal correctness. Scraping and modeling are important, but the system is only trustworthy if every feature and evaluation row reflects information available at prediction time.

Q: What are the main limitations?

A: No injury, camp, weight-cut, betting-market, or proprietary scouting data; sparse debut information; incomplete early stats; title fights and some weight classes are harder; and fighter ordering may carry subtle source bias.

Q: How would this scale?

A: For UFC-scale data, batch pandas plus PostgreSQL is sufficient. If usage grows, I would schedule jobs, materialize reporting views, add monitoring, partition prediction history, and expose a small API over the views.

Q: How do you know the model is not just memorizing?

A: The holdout is chronological, the final test period is future relative to training, there are baselines for comparison, and metrics are segmented by year, weight class, debut status, and confidence band.

Q: What trade-off did you make with model complexity?

A: I kept interpretable baselines and compared them to boosted trees. The final model is more complex than logistic regression but still explainable with feature importance/SHAP and much simpler operationally than sequence models.

Q: What would you improve first?

A: I would add scheduled orchestration and monitoring, strengthen deployment tests, materialize dashboard views if needed, and add richer external signals like market odds or injury/news data if licensing and ethics allow it.

10. Future Improvements

Product and Feature Improvements

- Add fighter comparison pages with key feature deltas and top drivers for each prediction.
- Expose confidence explanations: why a fight is high-confidence or a toss-up.
- Add alerting for upcoming cards that lack fighter profiles, stats coverage, or feature rows.
- Create a small API layer over the PostgreSQL reporting views if multiple clients need access.

Data and Modeling Improvements

- Add market odds as a benchmark or optional feature, while clearly separating pure-model predictions from market-informed predictions.
- Incorporate richer external context such as layoffs due to injuries, opponent changes, short-notice replacements, gym changes, and weigh-in information.
- Explore hierarchical or Bayesian models for uncertainty on sparse fighters and divisions.
- Add drift monitoring by year, division, and prediction confidence, and trigger model review when calibration worsens.

Engineering Improvements

- Use a scheduler such as GitHub Actions, cron, Airflow, or Prefect for repeatable scrape/load/score/review jobs.
- Add structured logs and metrics around scrape failures, row counts, feature null rates, prediction counts, and view freshness.
- Containerize the pipeline for consistent local and production execution.
- Add CI for migrations, transform tests, leakage tests, model artifact checks, and a small end-to-end pipeline smoke test.
- Version feature schemas and model artifacts more formally, including migration compatibility checks before scoring.

Best Interview Closing Summary

A strong concise summary: This is a data-first ML system. The core achievement is not only training a model, but designing the full loop: reliable acquisition, normalized storage, leakage-safe feature snapshots, calibrated probabilistic modeling, prediction history, and honest post-event evaluation.